

Estructuras de Datos y Algoritmos II

Facultad de Ingeniería, UNAM

Semestre 2026-1

Índice general

1	Introducción	1
1.1	¿Qué son los algoritmos?	1
1.1.1	Ejemplo: El problema del ordenamiento	1
1.1.2	¿Qué tipo de problemas resuelven los algoritmos?	1
1.2	Estructuras de datos	2
1.2.1	Estructuras de datos fundamentales	2
1.3	Análisis de algoritmos	2
1.3.1	Modelo RAM	2
1.3.2	Tiempo de ejecución	3
1.4	Notación asintótica	3
1.4.1	Notación O (O grande)	3
1.4.2	Notación Ω (Omega grande)	3
1.4.3	Notación Θ (Theta)	3
1.4.4	Complejidades comunes	3
1.5	Correctitud de algoritmos	3
1.5.1	Invariantes de ciclo	3
1.6	Diseño de algoritmos	4
1.6.1	Fuerza bruta	4
1.6.2	Divide y vencerás	4
1.6.3	Programación dinámica	4
1.6.4	Algoritmos voraces (Greedy)	4
1.6.5	Backtracking	4
1.6.6	Búsqueda local	5
1.7	Clases de Complejidad	5
1.7.1	Algoritmos factibles y no factibles	5
1.7.2	La clase P	5
1.7.3	La clase NP	6
1.7.4	Problemas NP -Completos	6
1.7.5	La pregunta P vs NP	6
1.7.6	Soluciones aproximadas	7
1.8	Elementos de Programación	7
1.8.1	Tipos de datos fundamentales	7
1.8.2	Estructuras de control	7
1.8.3	Manejo de memoria	8
1.9	Ejercicios	8
1.10	Referencias	9

2	Algoritmos de Ordenamiento	11
2.1	El problema del ordenamiento	11
2.1.1	Clasificación de algoritmos de ordenamiento	11
2.2	Insertion Sort	11
2.2.1	Pseudocódigo	12
2.2.2	Invariante de ciclo	12
2.2.3	Análisis de complejidad	13
2.3	Merge Sort	13
2.3.1	Pseudocódigo	13
2.3.2	Análisis de complejidad	15
2.4	Bubble Sort	17
2.4.1	Pseudocódigo	17
2.4.2	Optimización con bandera	17
2.4.3	Análisis de complejidad	18
2.5	Selection Sort	18
2.5.1	Pseudocódigo	19
2.5.2	Implementación en Python	19
2.5.3	Análisis de complejidad	19
2.6	Quick Sort	19
2.6.1	Pseudocódigo	20
2.6.2	Análisis de complejidad	20
2.7	Heap Sort	21
2.7.1	La estructura de datos Heap	21
2.7.2	Operación Max-Heapify	22
2.7.3	Algoritmo Heap Sort	23
2.8	Límite inferior para ordenamiento por comparaciones	23
2.9	Ordenamiento en tiempo lineal	23
2.9.1	Counting Sort	23
2.9.2	Radix Sort	24
2.10	Comparación de algoritmos	25
2.11	Ejercicios	25
2.11.1	Ejercicios Teóricos	25
2.11.2	Ejercicios de Programación	26
2.11.3	Problemas de LeetCode	27
2.11.4	Problemas Adicionales de Análisis	29
2.12	Referencias	29
2.13	Apéndice: Código de Soporte	29
2.13.1	Estructura del directorio	29
2.13.2	Módulo <code>test_utils.py</code>	29
2.13.3	Módulo <code>main.py</code>	31
2.13.4	Ejemplo: Comparación experimental	31
2.13.5	Verificación de correctitud	32
3	Algoritmos de Búsqueda	33
3.1	El problema de la búsqueda	33
3.2	Búsqueda Lineal (Secuencial)	33

3.2.1	Pseudocódigo	34
3.2.2	Análisis de complejidad	34
3.2.3	Invariante de ciclo	34
3.3	Búsqueda Binaria	34
3.3.1	Pseudocódigo iterativo	35
3.3.2	Pseudocódigo recursivo	35
3.3.3	Análisis de complejidad	35
3.3.4	Comparación: Lineal vs. Binaria	37
3.4	Tablas Hash	37
3.4.1	Direccionamiento directo	37
3.4.2	Funciones hash	37
3.4.3	Colisiones	38
3.5	Resolución de colisiones	38
3.5.1	Encadenamiento (Chaining)	38
3.5.2	Direccionamiento abierto (Open Addressing)	39
3.6	Hashing perfecto	40
3.7	Aplicaciones de tablas hash	41
3.8	Comparación de métodos de búsqueda	41
3.9	Ejercicios	41
3.10	Referencias	42
3.11	Apendice: Código de Soporte	42
3.11.1	Estructura del directorio	42
3.11.2	Módulo <code>main.py</code>	42
3.11.3	Comparación de algoritmos	44
3.11.4	Ejemplo: Comparación de rendimiento	44
3.11.5	Errores comunes	45

CAPÍTULO 1

Introducción

1.1 ¿Qué son los algoritmos?

Un **algoritmo** es un procedimiento computacional bien definido que toma algún valor, o conjunto de valores, como **entrada** y produce algún valor, o conjunto de valores, como **salida** en un intervalo de tiempo finito.

También podemos ver un algoritmo como una herramienta para resolver un **problema computacional** bien especificado. El enunciado del problema especifica en términos generales la relación deseada entre la entrada y la salida. El algoritmo describe un procedimiento computacional específico para lograr esa relación.

Definición 1.1 (Algoritmo). *Un algoritmo es una secuencia finita de instrucciones precisas para realizar una tarea, que dado un estado inicial, terminará en un estado final definido.*

1.1.1 Ejemplo: El problema del ordenamiento

El **problema del ordenamiento** se define formalmente como:

Problema del Ordenamiento

Entrada: Una secuencia de n números $\langle a_1, a_2, \dots, a_n \rangle$.

Salida: Una permutación (reordenamiento) $\langle a'_1, a'_2, \dots, a'_n \rangle$ de la secuencia de entrada tal que $a'_1 \leq a'_2 \leq \dots \leq a'_n$.

Por ejemplo, dada la secuencia de entrada $\langle 31, 41, 59, 26, 41, 58 \rangle$, un algoritmo de ordenamiento devuelve como salida la secuencia $\langle 26, 31, 41, 41, 58, 59 \rangle$.

A tal secuencia de entrada se le llama una **instancia** del problema del ordenamiento. En general, una **instancia de un problema** consiste en la entrada necesaria para calcular una solución al problema.

1.1.2 ¿Qué tipo de problemas resuelven los algoritmos?

El ordenamiento no es el único problema computacional para el cual se han desarrollado algoritmos. Las aplicaciones prácticas de los algoritmos son ubicuas e incluyen:

- **Búsqueda en Internet:** Motores de búsqueda utilizan algoritmos para encontrar páginas relevantes entre miles de millones de documentos.
- **Redes sociales:** Algoritmos determinan qué contenido mostrar, sugieren amigos y detectan comunidades.
- **Criptografía:** Protege información sensible usando algoritmos de encriptación basados en teoría de números.
- **Asignación de recursos:** Desde rutas de vehículos hasta asignación de frecuencias en telecomunicaciones.

- **Bioinformática:** Comparación de secuencias de ADN, predicción de estructuras de proteínas.
- **Inteligencia Artificial:** Aprendizaje automático, procesamiento de lenguaje natural, visión por computadora.

1.2 Estructuras de datos

Una **estructura de datos** es una forma de almacenar y organizar datos para facilitar el acceso y las modificaciones. Ninguna estructura de datos única funciona bien para todos los propósitos, por lo que es importante conocer las fortalezas y limitaciones de varias de ellas.

Definición 1.2 (Estructura de Datos). *Una estructura de datos es una colección de valores, las relaciones entre ellos, y las funciones u operaciones que se pueden aplicar a los datos.*

1.2.1 Estructuras de datos fundamentales

Arreglos

Un **arreglo** es una colección de elementos del mismo tipo almacenados en posiciones contiguas de memoria. Cada elemento se puede acceder directamente mediante un índice.

- **Acceso:** $O(1)$ - tiempo constante
- **Búsqueda:** $O(n)$ - tiempo lineal (sin ordenar)
- **Inserción/ Eliminación:** $O(n)$ - requiere desplazar elementos

Listas enlazadas

Una **lista enlazada** es una estructura donde cada elemento (nodo) contiene un dato y una referencia al siguiente elemento.

- **Acceso:** $O(n)$ - debe recorrer desde el inicio
- **Inserción/ Eliminación:** $O(1)$ - solo cambia referencias (si ya está en posición)

Pilas y Colas

- **Pila (Stack):** Estructura LIFO (Last In, First Out). Operaciones: push, pop.
- **Cola (Queue):** Estructura FIFO (First In, First Out). Operaciones: enqueue, dequeue.

1.3 Análisis de algoritmos

Antes de poder analizar un algoritmo, debemos tener un modelo de la tecnología de implementación que usaremos, incluyendo un modelo para los recursos de esa tecnología y sus costos.

1.3.1 Modelo RAM

Asumimos un modelo genérico de máquina de acceso aleatorio (**RAM**) de un solo procesador, en el cual las instrucciones se ejecutan una tras otra, sin operaciones concurrentes.

El modelo RAM contiene instrucciones comúnmente encontradas en computadoras reales:

- Aritméticas: suma, resta, multiplicación, división, residuo, piso, techo
- Movimiento de datos: cargar, almacenar, copiar
- Control: salto condicional e incondicional, llamada y retorno de subrutinas

Cada una de estas instrucciones toma una cantidad **constante** de tiempo.

1.3.2 Tiempo de ejecución

El **tiempo de ejecución** de un algoritmo sobre una entrada particular es el número de operaciones primitivas o “pasos” ejecutados.

Es conveniente definir la noción de paso de manera que sea lo más independiente posible de la máquina. Por ahora, adoptamos la siguiente perspectiva: se requiere una cantidad constante de tiempo para ejecutar cada línea de nuestro pseudocódigo.

1.4 Notación asintótica

Cuando observamos tamaños de entrada lo suficientemente grandes para hacer relevante solo el orden de crecimiento del tiempo de ejecución, estudiamos la **eficiencia asintótica** de los algoritmos.

1.4.1 Notación O (O grande)

La notación O caracteriza un **límite superior** en el comportamiento asintótico de una función. Es decir, acota qué tan rápido crece una función.

Definición 1.3 (Notación O grande). Para una función dada $g(n)$, denotamos por $O(g(n))$ al conjunto de funciones:

$$O(g(n)) = \{f(n) : \text{existen constantes positivas } c \text{ y } n_0 \text{ tales que } 0 \leq f(n) \leq c \cdot g(n) \text{ para todo } n \geq n_0\}$$

Escribimos $f(n) = O(g(n))$ para indicar que $f(n)$ es miembro del conjunto $O(g(n))$.

1.4.2 Notación Ω (Omega grande)

La notación Ω proporciona un **límite inferior** asintótico.

Definición 1.4 (Notación Omega grande).

$$\Omega(g(n)) = \{f(n) : \text{existen constantes positivas } c \text{ y } n_0 \text{ tales que } 0 \leq c \cdot g(n) \leq f(n) \text{ para todo } n \geq n_0\}$$

1.4.3 Notación Θ (Theta)

La notación Θ acota una función desde arriba y desde abajo asintóticamente.

Definición 1.5 (Notación Theta).

$$\Theta(g(n)) = \{f(n) : \text{existen constantes positivas } c_1, c_2 \text{ y } n_0 \text{ tales que } 0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \text{ para todo } n \geq n_0\}$$

1.4.4 Complejidades comunes

1.5 Correctitud de algoritmos

Un algoritmo es **correcto** si, para cada instancia de entrada, termina con la salida correcta. Decimos que un algoritmo correcto **resuelve** el problema computacional dado.

1.5.1 Invariantes de ciclo

Una técnica común para demostrar la correctitud de un algoritmo es usar **invariantes de ciclo**. Debemos demostrar tres cosas sobre un invariante de ciclo:

1. **Inicialización:** Es verdadero antes de la primera iteración del ciclo.
2. **Mantenimiento:** Si es verdadero antes de una iteración del ciclo, permanece verdadero antes de la siguiente iteración.
3. **Terminación:** Cuando el ciclo termina, el invariante nos da una propiedad útil que ayuda a mostrar que el algoritmo es correcto.

Notación	Nombre	Ejemplo
$O(1)$	Constante	Acceso a arreglo por índice
$O(\log n)$	Logarítmica	Búsqueda binaria
$O(n)$	Lineal	Búsqueda lineal
$O(n \log n)$	Linealítmica	Merge Sort, Quick Sort (promedio)
$O(n^2)$	Cuadrática	Bubble Sort, Insertion Sort
$O(n^3)$	Cúbica	Multiplicación de matrices (ingenua)
$O(2^n)$	Exponencial	Subconjuntos de un conjunto
$O(n!)$	Factorial	Permutaciones

Cuadro 1.1: Complejidades computacionales comunes

1.6 Diseño de algoritmos

Existen varios paradigmas para diseñar algoritmos:

1.6.1 Fuerza bruta

Examina todas las posibles soluciones y selecciona la mejor. Simple pero generalmente ineficiente.

Complejidad típica: Exponencial o factorial.

1.6.2 Divide y vencerás

Divide el problema en subproblemas más pequeños del mismo tipo, resuelve los subproblemas recursivamente y combina las soluciones.

Ejemplos: Merge Sort, Quick Sort, Búsqueda Binaria.

Recurrencia típica: $T(n) = aT(n/b) + f(n)$

1.6.3 Programación dinámica

Resuelve subproblemas superpuestos y almacena sus resultados para evitar recalcularlos.

Características:

- **Subestructura óptima:** La solución óptima contiene soluciones óptimas de subproblemas.
- **Subproblemas superpuestos:** Los mismos subproblemas se resuelven múltiples veces.

Ejemplos: Fibonacci, Knapsack, Camino más corto.

1.6.4 Algoritmos voraces (Greedy)

Construye una solución paso a paso, eligiendo en cada paso la opción localmente óptima.

Ejemplos: Algoritmo de Dijkstra, Árbol de expansión mínima (Prim, Kruskal).

1.6.5 Backtracking

Construye soluciones incrementalmente, abandonando candidatos (“retrocediendo”) tan pronto como determina que no pueden llevar a una solución válida.

Características:

- Explora el espacio de soluciones sistemáticamente.
- “Poda” ramas que no pueden llevar a solución.
- Útil para problemas de satisfacción de restricciones.

Ejemplos: N-Reinas, Sudoku, Laberintos.

```

1 def backtrack(solucion_parcial):
2     if es_solucion(solucion_parcial):
3         procesar_solucion(solucion_parcial)
4         return
5
6     for candidato in generar_candidatos(solucion_parcial):
7         if es_valido(candidato, solucion_parcial):
8             solucion_parcial.append(candidato)
9             backtrack(solucion_parcial)
10            solucion_parcial.pop() # Retroceder

```

1.6.6 Búsqueda local

Comienza con una solución inicial y la mejora iterativamente mediante modificaciones locales.

Algoritmos:

- **Hill Climbing:** Siempre se mueve hacia vecinos mejores.
- **Simulated Annealing:** Permite movimientos a soluciones peores con cierta probabilidad.
- **Algoritmos genéticos:** Evoluciona una población de soluciones.

1.7 Clases de Complejidad

1.7.1 Algoritmos factibles y no factibles

Definición 1.6 (Algoritmo de tiempo polinomial). *Un algoritmo es de **tiempo polinomial** si su complejidad temporal es $O(n^k)$ para alguna constante k .*

Los algoritmos de tiempo polinomial se consideran **factibles** o **eficientes**. Los algoritmos exponenciales ($O(2^n)$, $O(n!)$) son generalmente **no factibles** para entradas grandes.

Complejidad	$n = 20$	$n = 50$	$n = 100$
$O(n)$	20	50	100
$O(n^2)$	400	2,500	10,000
$O(n^3)$	8,000	125,000	1,000,000
$O(2^n)$	1,048,576	10^{15}	10^{30}
$O(n!)$	10^{18}	10^{64}	10^{157}

Cuadro 1.2: Crecimiento de diferentes complejidades (número de operaciones).

1.7.2 La clase P

Definición 1.7 (Clase P). *P es el conjunto de problemas de decisión que pueden ser resueltos por un algoritmo determinista en tiempo polinomial.*

Ejemplos en P:

- Ordenamiento
- Búsqueda en grafos
- Camino más corto
- Flujo máximo

1.7.3 La clase NP

Definición 1.8 (Clase NP). *NP* (Nondeterministic Polynomial) es el conjunto de problemas de decisión cuyas soluciones pueden ser **verificadas** en tiempo polinomial.

Es decir, si alguien te da una supuesta solución, puedes verificar si es correcta en tiempo polinomial.

Ejemplos en NP:

- **SAT:** ¿Existe una asignación que satisfaga una fórmula booleana?
- **Clique:** ¿Existe un clique de tamaño k en un grafo?
- **Subset Sum:** ¿Existe un subconjunto que sume exactamente S ?

Nota: $P \subseteq NP$ (todo lo que se puede resolver rápido, se puede verificar rápido).

1.7.4 Problemas NP-Completos

Definición 1.9 (NP-Completo). *Un problema es NP-Completo si:*

1. Está en NP.
2. Todo problema en NP se puede reducir a él en tiempo polinomial.

Los problemas NP-Completos son los “más difíciles” de NP. Si encontramos un algoritmo polinomial para cualquier problema NP-Completo, entonces $P = NP$.

Problemas NP-Completos famosos:

- SAT (Satisfacibilidad booleana)
- 3-SAT
- Clique
- Vertex Cover
- Hamiltonian Path/Cycle
- Traveling Salesman Problem (TSP) - versión decisión
- Knapsack (0/1)
- Graph Coloring

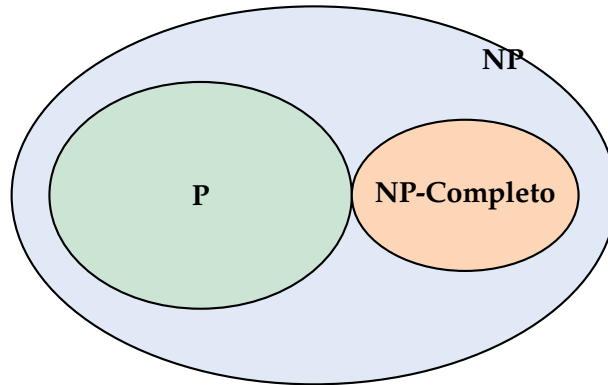
1.7.5 La pregunta P vs NP

La pregunta “¿Es $P = NP$?” es uno de los problemas abiertos más importantes en ciencias de la computación y matemáticas. Es uno de los **Problemas del Milenio** con un premio de \$1,000,000.

Si $P = NP$: Muchos problemas actualmente difíciles serían fáciles. La criptografía moderna colapsaría.

Si $P \neq NP$: Existen problemas inherentemente difíciles que nunca podrán resolverse eficientemente.

Consenso: La mayoría de los investigadores creen que $P \neq NP$.



(Asumiendo $P \neq NP$)

Figura 1.1: Relación entre las clases P, NP y NP-Completo.

1.7.6 Soluciones aproximadas

Cuando enfrentamos problemas NP-Completo, podemos usar:

1. **Algoritmos de aproximación:** Garantizan soluciones dentro de un factor de la óptima.
2. **Heurísticas:** Sin garantías, pero funcionan bien en la práctica.
3. **Algoritmos aleatorizados:** Usan aleatoriedad para encontrar buenas soluciones.
4. **Casos especiales:** Algunos casos del problema pueden ser tratables.
5. **Algoritmos parametrizados:** Eficientes cuando algún parámetro es pequeño.

1.8 Elementos de Programación

1.8.1 Tipos de datos fundamentales

Los tipos de datos primitivos son los bloques básicos de construcción:

Tipo	Python	Tamaño típico	Ejemplo
Entero	int	Variable	42, -17
Real	float	64 bits	3.14159
Booleano	bool	1 bit (lógico)	True, False
Caracter	str (longitud 1)	Variable	'a', 'Z'
Cadena	str	Variable	"Hola"

Cuadro 1.3: Tipos de datos primitivos.

1.8.2 Estructuras de control

Secuencia: Instrucciones ejecutadas en orden.

Selección (condicionales):

```

1 if condicion:
2     # bloque si verdadero
3 elif otra_condicion:
```

```

4     # bloque alternativo
5 else:
6     # bloque por defecto

```

Iteración (ciclos):

```

1 # Ciclo for - iteracion definida
2 for i in range(n):
3     # se ejecuta n veces
4
5 # Ciclo while - iteracion indefinida
6 while condicion:
7     # mientras condicion sea verdadera

```

1.8.3 Manejo de memoria

En Python, la memoria es manejada automáticamente mediante:

- **Asignación dinámica:** Los objetos se crean en el heap.
- **Recolección de basura:** Objetos no referenciados son liberados automáticamente.
- **Conteo de referencias:** Python cuenta cuántas referencias apuntan a cada objeto.

Paso por referencia vs. valor:

```

1 # En Python, todo es por referencia al objeto
2 # Pero los inmutables (int, str, tuple) no pueden modificarse
3
4 def modificar(lista, numero):
5     lista.append(4)    # Modifica la lista original
6     numero = numero + 1 # Crea nuevo objeto, no modifica original
7
8 mi_lista = [1, 2, 3]
9 mi_numero = 10
10 modificar(mi_lista, mi_numero)
11 print(mi_lista)    # [1, 2, 3, 4] - modificada
12 print(mi_numero)   # 10 - sin cambios

```

1.9 Ejercicios

1. Ordena las siguientes funciones por orden de crecimiento: n , $\sqrt{n}$, $n^{1.5}$, n^2 , $n \log n$, $n \log \log n$, $n \log^2 n$, $n \log(n^2)$, 2^n , $2^{n/2}$, $n!$.
2. Demuestra que $n^2 + 100n + 1000 = O(n^2)$.
3. ¿Es $2^{n+1} = O(2^n)$? ¿Es $2^{2n} = O(2^n)$?
4. Para cada función $f(n)$ y tiempo t en la siguiente tabla, determina el tamaño máximo n de un problema que puede resolverse en tiempo t , asumiendo que el algoritmo tarda $f(n)$ microsegundos.
5. Describe un algoritmo que, dado un conjunto S de n enteros y otro entero x , determine si existen dos elementos en S cuya suma sea exactamente x .

1.10 Referencias

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3ra ed.). MIT Press.
- Knuth, D. E. (1997). *The Art of Computer Programming*. Addison-Wesley.
- Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4ta ed.). Addison-Wesley.

CAPÍTULO 2

Algoritmos de Ordenamiento

El problema del ordenamiento es fundamental en ciencias de la computación. Un algoritmo de ordenamiento reorganiza los elementos de un arreglo o lista de acuerdo a una relación de orden. En este capítulo estudiaremos varios algoritmos clásicos, analizando su funcionamiento, correctitud y complejidad.

2.1 El problema del ordenamiento

Recordemos la definición formal del problema:

Problema del Ordenamiento

Entrada: Una secuencia de n números $\langle a_1, a_2, \dots, a_n \rangle$.

Salida: Una permutación (reordenamiento) $\langle a'_1, a'_2, \dots, a'_n \rangle$ de la secuencia de entrada tal que $a'_1 \leq a'_2 \leq \dots \leq a'_n$.

Los números que deseamos ordenar también se conocen como **claves**. Aunque conceptualmente estamos ordenando una secuencia, la entrada viene en forma de un arreglo con n elementos.

2.1.1 Clasificación de algoritmos de ordenamiento

Los algoritmos de ordenamiento se pueden clasificar de varias formas:

- **Por comparación vs. sin comparación:** Los algoritmos basados en comparaciones (como Merge Sort, Quick Sort) solo usan comparaciones entre elementos. Los algoritmos sin comparación (como Counting Sort, Radix Sort) explotan propiedades de los datos.
- **Estables vs. inestables:** Un algoritmo es **estable** si mantiene el orden relativo de elementos con claves iguales.
- **In-place vs. con memoria auxiliar:** Un algoritmo es **in-place** si usa solo $O(1)$ memoria adicional.
- **Internos vs. externos:** El ordenamiento **interno** asume que todos los datos caben en memoria principal. El **externo** maneja datos que residen en almacenamiento secundario.

2.2 Insertion Sort

Insertion Sort es un algoritmo simple y eficiente para ordenar un número pequeño de elementos. Funciona de la manera en que muchas personas ordenan una mano de cartas:

1. Comenzamos con la mano izquierda vacía y las cartas boca abajo en la mesa.
2. Tomamos una carta a la vez de la mesa y la insertamos en la posición correcta en la mano izquierda.
3. Para encontrar la posición correcta, comparamos la carta con cada una de las cartas ya en la mano, de derecha a izquierda.

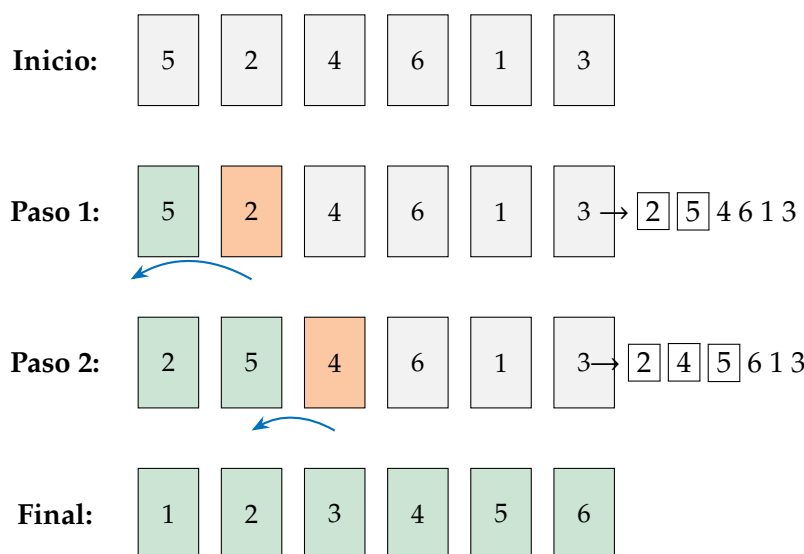


Figura 2.1: Funcionamiento de Insertion Sort. Los elementos verdes están ordenados, el naranja es el elemento actual siendo insertado.

2.2.1 Pseudocódigo

Algorithm 1 Insertion-Sort(A, n)

```

1: for  $j = 2$  to  $n$  do
2:    $key = A[j]$ 
3:   {Insertar  $A[j]$  en la secuencia ordenada  $A[1..j - 1]$ }
4:    $i = j - 1$ 
5:   while  $i > 0$  and  $A[i] > key$  do
6:      $A[i + 1] = A[i]$ 
7:      $i = i - 1$ 
8:   end while
9:    $A[i + 1] = key$ 
10: end for

```

2.2.2 Invariante de ciclo

Para demostrar que Insertion Sort es correcto, usamos el siguiente invariante de ciclo:

Invariante: Al inicio de cada iteración del ciclo **for**, el subarreglo $A[1..j - 1]$ consiste en los elementos originalmente en $A[1..j - 1]$, pero en orden.

- **Inicialización:** Antes de la primera iteración ($j = 2$), el subarreglo $A[1..1]$ consiste en un solo elemento, que trivialmente está ordenado.
- **Mantenimiento:** El cuerpo del ciclo mueve $A[j - 1]$, $A[j - 2]$, etc. una posición a la derecha hasta encontrar la posición correcta para $A[j]$ (líneas 5-8), momento en el cual inserta el valor de $A[j]$ (línea 9). El subarreglo $A[1..j]$ queda ordenado.

- **Terminación:** El ciclo termina cuando $j = n + 1$. Sustituyendo $n + 1$ por j en el invariante, tenemos que $A[1..n]$ está ordenado.

2.2.3 Análisis de complejidad

Teorema 2.1. *Insertion Sort tiene complejidad $O(n^2)$ en el peor caso y $O(n)$ en el mejor caso.*

Demostración. El ciclo **for** externo se ejecuta $n - 1$ veces. En el **peor caso** (arreglo en orden inverso), el ciclo **while** interno se ejecuta $j - 1$ veces para cada j :

$$\sum_{j=2}^n (j - 1) = 1 + 2 + \dots + (n - 1) = \frac{n(n - 1)}{2} = \Theta(n^2)$$

En el **mejor caso** (arreglo ya ordenado), el ciclo **while** nunca se ejecuta, resultando en $\Theta(n)$. $\square$

2.3 Merge Sort

Merge Sort es un algoritmo que sigue el paradigma de **divide y vencerás**:

1. **Dividir:** Divide la secuencia de n elementos en dos subsecuencias de $n/2$ elementos cada una.
2. **Conquistar:** Ordena las dos subsecuencias recursivamente usando Merge Sort.
3. **Combinar:** Mezcla las dos subsecuencias ordenadas para producir la respuesta ordenada.

2.3.1 Pseudocódigo

Algorithm 2 Merge-Sort(A, p, r)

```

1: if  $p \geq r$  then
2:   return {Caso base: 0 o 1 elementos}
3: end if
4:  $q = \lfloor (p + r)/2 \rfloor$  {Punto medio}
5: Merge-Sort( $A, p, q$ ) {Ordenar mitad izquierda}
6: Merge-Sort( $A, q + 1, r$ ) {Ordenar mitad derecha}
7: Merge( $A, p, q, r$ ) {Combinar las mitades}
```

La operación clave es el procedimiento **MERGE**, que combina dos subarreglos ordenados $A[p..q]$ y $A[q + 1..r]$ en un solo subarreglo ordenado:

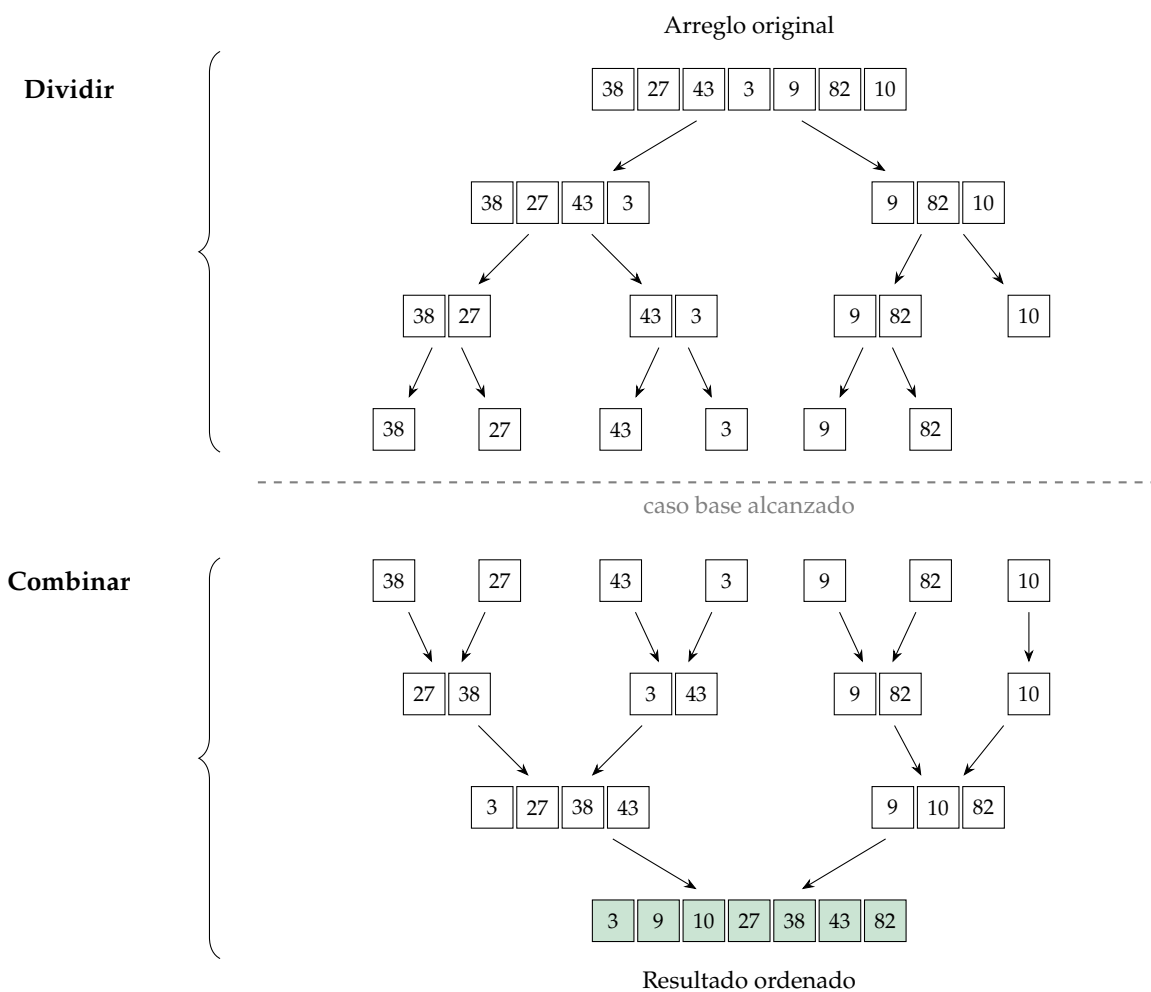


Figura 2.2: Merge Sort: fase de división (arriba) y fase de combinación (abajo). Los elementos se dividen recursivamente hasta llegar al caso base, luego se combinan en orden.

Algorithm 3 Merge(A, p, q, r)

```

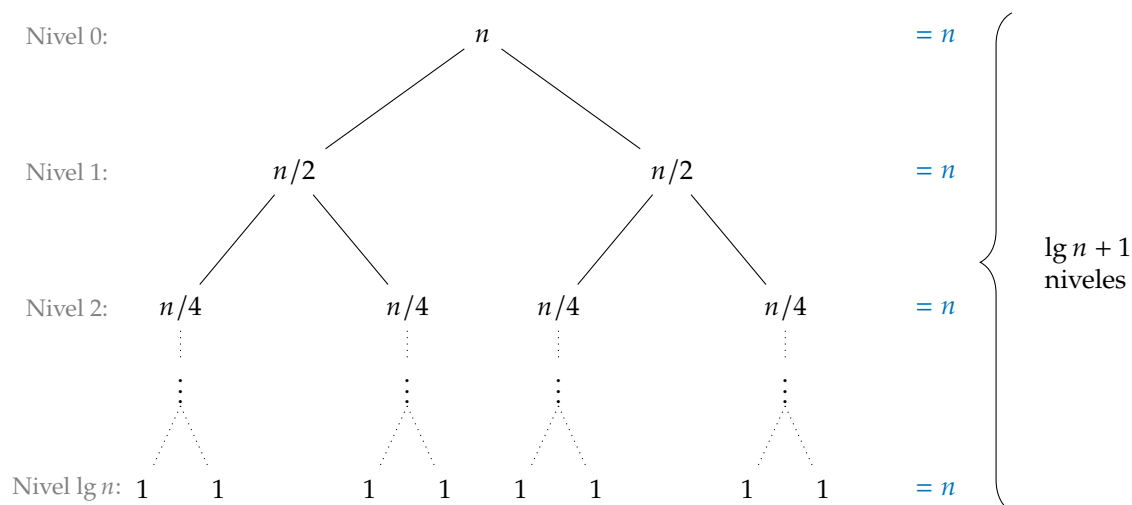
1:  $n_L = q - p + 1$  {Longitud de  $A[p..q]$ }
2:  $n_R = r - q$  {Longitud de  $A[q + 1..r]$ }
3: Crear arreglos  $L[1..n_L]$  y  $R[1..n_R]$ 
4: for  $i = 1$  to  $n_L$  do
5:    $L[i] = A[p + i - 1]$ 
6: end for
7: for  $j = 1$  to  $n_R$  do
8:    $R[j] = A[q + j]$ 
9: end for
10:  $i = 1, j = 1, k = p$ 
11: while  $i \leq n_L$  and  $j \leq n_R$  do
12:   if  $L[i] \leq R[j]$  then
13:      $A[k] = L[i]; i = i + 1$ 
14:   else
15:      $A[k] = R[j]; j = j + 1$ 
16:   end if
17:    $k = k + 1$ 
18: end while
19: while  $i \leq n_L$  do
20:    $A[k] = L[i]; i = i + 1; k = k + 1$ 
21: end while
22: while  $j \leq n_R$  do
23:    $A[k] = R[j]; j = j + 1; k = k + 1$ 
24: end while

```

2.3.2 Análisis de complejidad

El tiempo de ejecución de Merge Sort está dado por la recurrencia:

$$T(n) = \begin{cases} \Theta(1) & \text{si } n = 1 \\ 2T(n/2) + \Theta(n) & \text{si } n > 1 \end{cases}$$



$$\text{Total: } (\lg n + 1) \cdot n = n \lg n + n = \Theta(n \lg n)$$

(n hojas en el último nivel, cada una con costo $c = 1$)

Figura 2.3: Árbol de recurrencia para Merge Sort. En cada nivel el costo total es n . Como hay $\lg n + 1$ niveles, el costo total es $\Theta(n \lg n)$.

¿Por qué la altura del árbol es $\lg n$?

En cada nivel de recursión, el tamaño del problema se divide a la mitad:

Nivel	Tamaño del subproblema
0	n
1	$n/2$
2	$n/4 = n/2^2$
3	$n/8 = n/2^3$
$\vdots$	$\vdots$
k	$n/2^k$

La recursión termina cuando el tamaño del subproblema llega a 1 (caso base). Buscamos el nivel k donde:

$$\frac{n}{2^k} = 1 \Rightarrow n = 2^k \Rightarrow k = \log_2 n = \lg n$$

Por lo tanto, el árbol tiene $\lg n$ niveles desde el nivel 1 hasta las hojas. Contando el nivel 0 (la raíz), hay $\lg n + 1$ niveles en total.

Ejemplo 2.1. Si $n = 8$:

- Nivel 0: tamaño 8
- Nivel 1: tamaño 4
- Nivel 2: tamaño 2
- Nivel 3: tamaño 1 (caso base)

$Altura = \lg 8 = 3$ niveles, más la raíz = 4 niveles totales.

Teorema 2.2. Merge Sort tiene complejidad $\Theta(n \lg n)$ en todos los casos.

Demostración. El árbol de recurrencia tiene $\lg n + 1$ niveles (ya que dividimos a la mitad en cada nivel hasta llegar a subproblemas de tamaño 1). En cada nivel, el trabajo total es n (la suma de los costos de todos los nodos en ese nivel). Por lo tanto:

$$T(n) = n \cdot (\lg n + 1) = n \lg n + n = \Theta(n \lg n)$$

□

2.4 Bubble Sort

Bubble Sort es uno de los algoritmos de ordenamiento más simples. Funciona intercambiando repetidamente elementos adyacentes que están en el orden incorrecto. El nombre proviene de la forma en que los elementos más grandes “burbujean” hacia el final del arreglo.

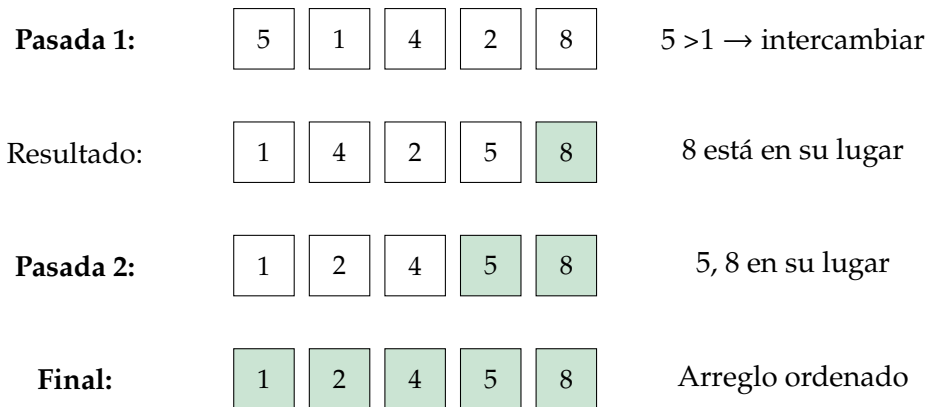


Figura 2.4: Bubble Sort: en cada pasada, el elemento más grande “burbujea” a su posición final.

2.4.1 Pseudocódigo

Algorithm 4 Bubble-Sort(A, n)

```

1: for  $i = 1$  to  $n - 1$  do
2:   for  $j = 1$  to  $n - i$  do
3:     if  $A[j] > A[j + 1]$  then
4:       intercambiar  $A[j]$  con  $A[j + 1]$ 
5:     end if
6:   end for
7: end for
```

2.4.2 Optimización con bandera

Podemos optimizar Bubble Sort detectando si en una pasada no hubo intercambios, lo que indica que el arreglo ya está ordenado:

Algorithm 5 Bubble-Sort-Optimizado(A, n)

```

1: for  $i = 1$  to  $n - 1$  do
2:    $swapped = \text{false}$ 
3:   for  $j = 1$  to  $n - i$  do
4:     if  $A[j] > A[j + 1]$  then
5:       intercambiar  $A[j]$  con  $A[j + 1]$ 
6:        $swapped = \text{true}$ 
7:     end if
8:   end for
9:   if  $not\ swapped$  then
10:    break {Ya está ordenado}
11:   end if
12: end for

```

2.4.3 Análisis de complejidad

- **Peor caso:** $O(n^2)$ - arreglo en orden inverso
- **Mejor caso:** $O(n)$ - arreglo ya ordenado (con optimización)
- **Caso promedio:** $O(n^2)$

2.5 Selection Sort

Selection Sort funciona seleccionando repetidamente el elemento mínimo de la parte no ordenada y moviéndolo al principio.

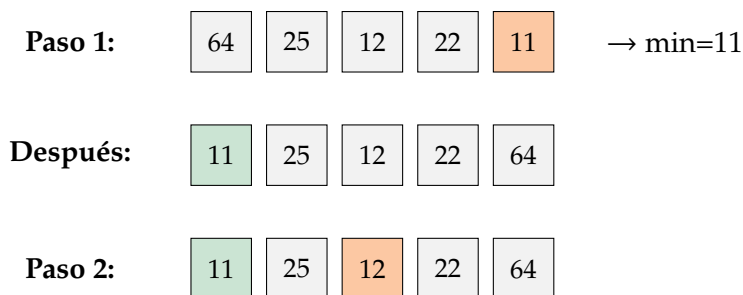


Figura 2.5: Selection Sort: en cada paso, se encuentra el mínimo de la parte no ordenada.

2.5.1 Pseudocódigo

Algorithm 6 Selection-Sort(A, n)

```

1: for  $i = 1$  to  $n - 1$  do
2:    $min\_idx = i$ 
3:   for  $j = i + 1$  to  $n$  do
4:     if  $A[j] < A[min\_idx]$  then
5:        $min\_idx = j$ 
6:     end if
7:   end for
8:   intercambiar  $A[i]$  con  $A[min\_idx]$ 
9: end for

```

2.5.2 Implementación en Python

```

1 def selection_sort(arr):
2     n = len(arr)
3     for i in range(n - 1):
4         min_idx = i
5         for j in range(i + 1, n):
6             if arr[j] < arr[min_idx]:
7                 min_idx = j
8         arr[i], arr[min_idx] = arr[min_idx], arr[i]
9     return arr

```

2.5.3 Análisis de complejidad

- **Comparaciones:** Siempre $\frac{n(n-1)}{2} = O(n^2)$
- **Intercambios:** Exactamente $n - 1 = O(n)$
- **Mejor, peor y promedio:** $O(n^2)$

Ventaja: Minimiza el número de intercambios (útil cuando los intercambios son costosos).

Desventaja: No es adaptativo (siempre hace $O(n^2)$ comparaciones).

2.6 Quick Sort

Quick Sort es un algoritmo de ordenamiento que, al igual que Merge Sort, aplica el paradigma de divide y vencerás. Sin embargo, a diferencia de Merge Sort, todo el trabajo real se hace en la fase de división:

1. **Dividir:** Particionar el arreglo $A[p..r]$ en dos subarreglos $A[p..q - 1]$ y $A[q + 1..r]$ tales que cada elemento de $A[p..q - 1]$ es menor o igual que $A[q]$, que a su vez es menor o igual que cada elemento de $A[q + 1..r]$.
2. **Conquistar:** Ordenar los dos subarreglos recursivamente.
3. **Combinar:** No se requiere trabajo; el arreglo ya está ordenado.

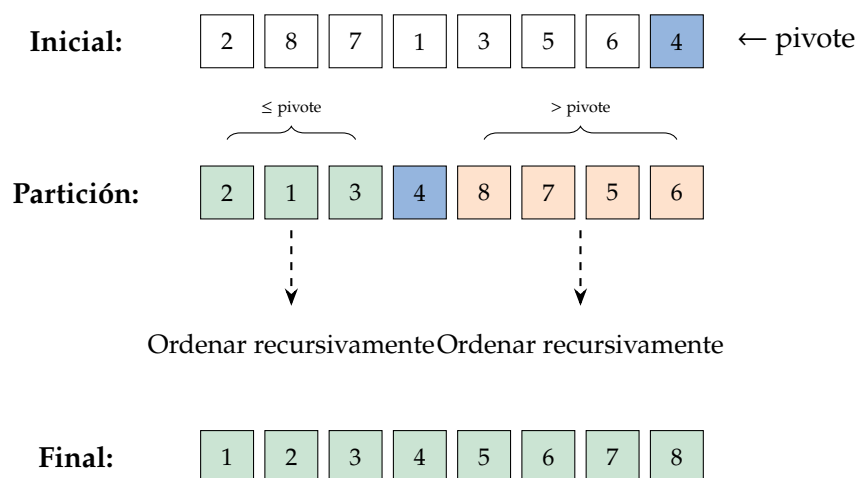


Figura 2.6: Quick Sort: el pivote (4) divide el arreglo en elementos menores y mayores.

2.6.1 Pseudocódigo

Algorithm 7 Quick-Sort(A, p, r)

```

1: if  $p < r$  then
2:    $q = \text{Partition}(A, p, r)$ 
3:   Quick-Sort( $A, p, q - 1$ )
4:   Quick-Sort( $A, q + 1, r$ )
5: end if

```

La clave de Quick Sort es el procedimiento PARTITION:

Algorithm 8 Partition(A, p, r)

```

1:  $x = A[r]$  {El pivote}
2:  $i = p - 1$ 
3: for  $j = p$  to  $r - 1$  do
4:   if  $A[j] \leq x$  then
5:      $i = i + 1$ 
6:     intercambiar  $A[i]$  con  $A[j]$ 
7:   end if
8: end for
9: intercambiar  $A[i + 1]$  con  $A[r]$ 
10: return  $i + 1$ 

```

2.6.2 Análisis de complejidad

El rendimiento de Quick Sort depende de qué tan balanceadas sean las particiones:

- **Peor caso:** $\Theta(n^2)$ - cuando las particiones son maximalmente desbalanceadas (ej. arreglo ya ordenado con pivote al final).
- **Mejor caso:** $\Theta(n \lg n)$ - cuando las particiones son perfectamente balanceadas.

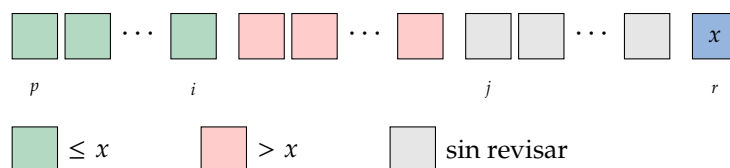
Invariante de Partition:

Figura 2.7: Invariante del procedimiento Partition: elementos a la izquierda de i son $\leq x$, elementos entre i y j son $> x$.

- **Caso promedio:** $\Theta(n \lg n)$ - asumiendo que todas las permutaciones de entrada son igualmente probables.

Teorema 2.3. El tiempo de ejecución esperado de Quick Sort es $O(n \lg n)$.

2.7 Heap Sort

Heap Sort introduce una estructura de datos llamada **heap** (montículo), que es útil tanto para ordenamiento como para implementar colas de prioridad.

2.7.1 La estructura de datos Heap

Un **heap binario** es un arreglo que podemos ver como un árbol binario casi completo. Cada nodo del árbol corresponde a un elemento del arreglo.

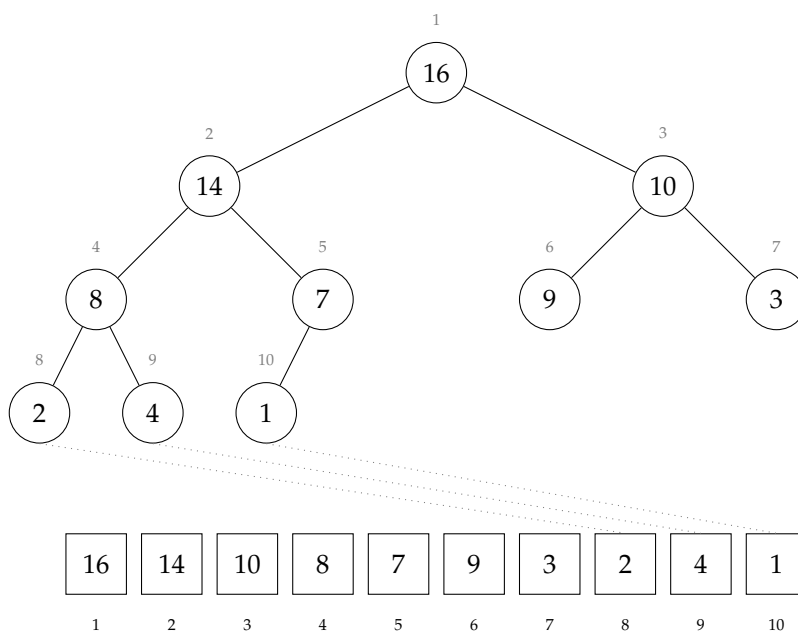


Figura 2.8: Un max-heap visto como árbol binario y como arreglo. El valor de cada nodo es $\geq$ que los valores de sus hijos.

Para un nodo en el índice i :

- **Padre:** $\lfloor i/2 \rfloor$

- **Hijo izquierdo:** $2i$
- **Hijo derecho:** $2i + 1$

Definición 2.1 (Propiedad de Max-Heap). *Para todo nodo i diferente de la raíz: $A[\text{Parent}(i)] \geq A[i]$. Es decir, el valor de un nodo es a lo más el valor de su padre. El elemento más grande está en la raíz.*

2.7.2 Operación Max-Heapify

MAX-HEAPIFY es el procedimiento clave para mantener la propiedad de max-heap. Asume que los subárboles izquierdo y derecho de un nodo i son max-heaps, pero $A[i]$ podría ser menor que sus hijos.

Algorithm 9 Max-Heapify(A, i)

```

1:  $l = \text{Left}(i)$ 
2:  $r = \text{Right}(i)$ 
3: if  $l \leq A.\text{heap-size}$  and  $A[l] > A[i]$  then
4:    $\text{largest} = l$ 
5: else
6:    $\text{largest} = i$ 
7: end if
8: if  $r \leq A.\text{heap-size}$  and  $A[r] > A[\text{largest}]$  then
9:    $\text{largest} = r$ 
10: end if
11: if  $\text{largest} \neq i$  then
12:   intercambiar  $A[i]$  con  $A[\text{largest}]$ 
13:   Max-Heapify( $A, \text{largest}$ )
14: end if

```

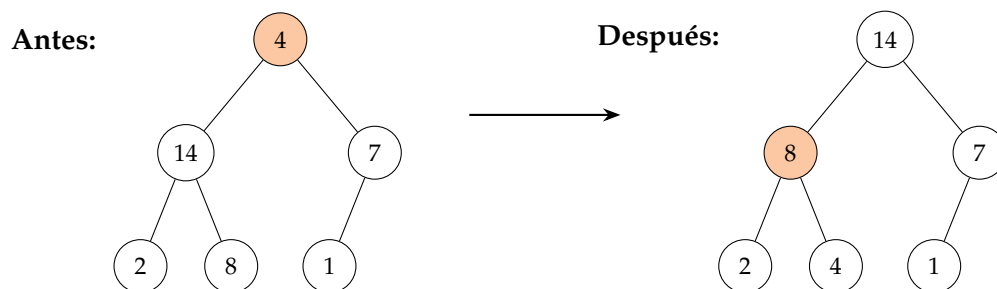


Figura 2.9: Max-Heapify: el valor 4 “baja” intercambiándose con el mayor de sus hijos hasta restaurar la propiedad de heap.

2.7.3 Algoritmo Heap Sort

Algorithm 10 Heap-Sort(A, n)

```

1: Build-Max-Heap( $A, n$ ) {Construir el heap}
2: for  $i = n$  downto 2 do
3:   intercambiar  $A[1]$  con  $A[i]$  {Mover máximo al final}
4:    $A.heap-size = A.heap-size - 1$ 
5:   Max-Heapify( $A, 1$ ) {Restaurar propiedad}
6: end for

```

Teorema 2.4. *Heap Sort tiene complejidad $O(n \lg n)$ en todos los casos.*

2.8 Límite inferior para ordenamiento por comparaciones

¿Podemos hacer algo mejor que $O(n \lg n)$? Para algoritmos basados en comparaciones, la respuesta es no.

Teorema 2.5. *Cualquier algoritmo de ordenamiento basado en comparaciones requiere $\Omega(n \lg n)$ comparaciones en el peor caso.*

Demostración. Un algoritmo basado en comparaciones puede verse como un árbol de decisión binario. Con $n!$ permutaciones posibles, el árbol debe tener al menos $n!$ hojas. Un árbol binario con L hojas tiene altura al menos $\lg L$. Por lo tanto:

$$h \geq \lg(n!) = \Omega(n \lg n)$$

usando la aproximación de Stirling: $n! > (n/e)^n$. □

2.9 Ordenamiento en tiempo lineal

Los algoritmos que no se basan en comparaciones pueden superar el límite de $\Omega(n \lg n)$.

2.9.1 Counting Sort

Counting Sort asume que los n elementos de entrada son enteros en el rango $[0, k]$. Cuando $k = O(n)$, el ordenamiento corre en tiempo $\Theta(n)$.

Algorithm 11 Counting-Sort(A, B, n, k)

```

1: Crear arreglo  $C[0..k]$ 
2: for  $i = 0$  to  $k$  do
3:    $C[i] = 0$ 
4: end for
5: for  $j = 1$  to  $n$  do
6:    $C[A[j]] = C[A[j]] + 1$  { $C[i] = \#$  elementos iguales a  $i$ }
7: end for
8: for  $i = 1$  to  $k$  do
9:    $C[i] = C[i] + C[i - 1]$  { $C[i] = \#$  elementos  $\leq i$ }
10: end for
11: for  $j = n$  downto 1 do
12:    $B[C[A[j]]] = A[j]$ 
13:    $C[A[j]] = C[A[j]] - 1$ 
14: end for

```

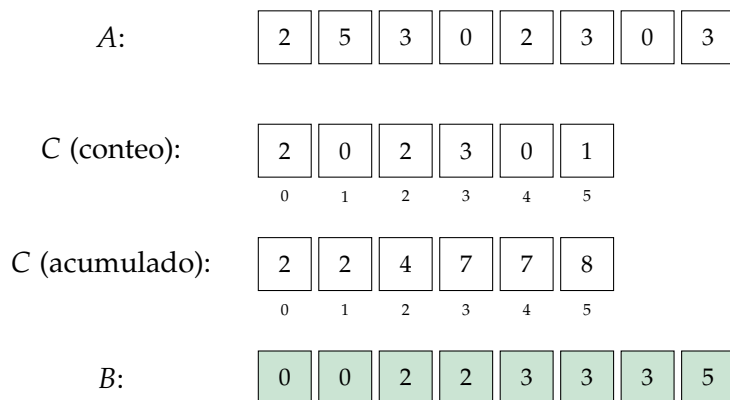


Figura 2.10: Counting Sort: $C[i]$ cuenta cuántos elementos son $\leq i$, lo que determina la posición final.

Counting Sort es **estable**: elementos con el mismo valor aparecen en el arreglo de salida en el mismo orden que en el de entrada.

2.9.2 Radix Sort

Radix Sort ordena números de d dígitos procesando un dígito a la vez, del menos significativo al más significativo.

Algorithm 12 Radix-Sort(A, d)

- 1: **for** $i = 1$ **to** d **do**
 - 2: usar un ordenamiento estable para ordenar A por el dígito i
 - 3: **end for**
-

Original	Por unidades	Por decenas	Por centenas
329	720	720	329
457	355	329	355
657	436	436	436
839	457	839	457
436	657	355	657
720	329	457	720
355	839	657	839
	→	→	→

Figura 2.11: Radix Sort ordena números de 3 dígitos procesando del dígito menos significativo al más significativo.

Teorema 2.6. *Dado n números de d dígitos donde cada dígito puede tomar hasta k valores posibles, Radix Sort ordena correctamente en tiempo $\Theta(d(n + k))$.*

Algoritmo	Mejor	Promedio	Peor	Estable
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Sí
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Sí
Merge Sort	$O(n \lg n)$	$O(n \lg n)$	$O(n \lg n)$	Sí
Quick Sort	$O(n \lg n)$	$O(n \lg n)$	$O(n^2)$	No
Heap Sort	$O(n \lg n)$	$O(n \lg n)$	$O(n \lg n)$	No
Counting Sort	$O(n + k)$	$O(n + k)$	$O(n + k)$	Sí
Radix Sort	$O(dn)$	$O(dn)$	$O(dn)$	Sí

Cuadro 2.1: Comparación de complejidades temporales de algoritmos de ordenamiento.

2.10 Comparación de algoritmos

2.11 Ejercicios

2.11.1 Ejercicios Teóricos

Insertion Sort

1. Ilustra la operación de Insertion Sort sobre el arreglo $A = \langle 31, 41, 59, 26, 41, 58 \rangle$.
2. Reescribe el procedimiento Insertion-Sort para ordenar en orden no creciente en lugar de no decreciente.
3. Considera un arreglo de n elementos donde los primeros $n - 1$ elementos ya están ordenados. ¿Cuál es la complejidad de Insertion Sort en este caso? Justifica tu respuesta.
4. **Búsqueda binaria en Insertion Sort:** Modifica Insertion Sort para usar búsqueda binaria al encontrar la posición de inserción. ¿Mejora esto la complejidad temporal? ¿Por qué sí o por qué no?

Merge Sort

5. Usando la Figura 2.2 como modelo, ilustra la operación de Merge Sort sobre el arreglo $A = \langle 3, 41, 52, 26, 38, 57, 9, 49 \rangle$.
6. Escriba la recurrencia para el tiempo de ejecución de Merge Sort cuando el arreglo se divide en tercios en lugar de mitades. ¿Cuál es la solución de esta recurrencia?
7. **Inversiones:** Una *inversión* en un arreglo $A[1..n]$ es un par (i, j) tal que $i < j$ y $A[i] > A[j]$. Modifica Merge Sort para contar el número de inversiones en tiempo $\Theta(n \lg n)$.
8. Demuestra que cualquier algoritmo de ordenamiento basado en comparaciones requiere $\Omega(n \lg n)$ comparaciones en el peor caso.

Bubble Sort

9. Demuestre que la versión optimizada de Bubble Sort tiene tiempo de ejecución $\Omega(n)$ en el mejor caso.
10. ¿Cuántos intercambios realiza Bubble Sort en el peor caso para un arreglo de n elementos? Demuéstralo.
11. **Cocktail Shaker Sort:** Investiga e implementa una variante de Bubble Sort llamada “Cocktail Shaker Sort” que alterna la dirección de las pasadas. ¿Mejora el rendimiento en algún caso?

Quick Sort

12. Ilustre la operación de Partition sobre el arreglo $A = \langle 13, 19, 9, 5, 12, 8, 7, 4, 21, 2, 6, 11 \rangle$.
13. ¿Cuál es el tiempo de ejecución de Quick Sort cuando todos los elementos del arreglo tienen el mismo valor?
14. **Mediana de tres:** Implementa una versión de Quick Sort que elija el pivote como la mediana de tres elementos: el primero, el del medio y el último. ¿Cómo afecta esto al rendimiento en arreglos ya ordenados?
15. Demuestra que la profundidad de la pila de recursión de Quick Sort en el peor caso es $\Theta(n)$. ¿Cómo se puede modificar Quick Sort para garantizar que la profundidad sea $O(\lg n)$?
16. **Quick Sort con 3 particiones:** Diseña una versión de Quick Sort que particione el arreglo en tres partes: elementos menores que el pivote, iguales al pivote, y mayores que el pivote. ¿Cuál es la ventaja de esta versión?

Heap Sort

17. Usando la Figura 2.8 como modelo, ilustre la operación de Build-Max-Heap sobre el arreglo $A = \langle 5, 3, 17, 10, 84, 19, 6, 22, 9 \rangle$.
18. Demuestra que hay a lo más $\lceil n/2^{h+1} \rceil$ nodos de altura h en cualquier heap de n elementos.
19. **Min-Heap:** Escribe los procedimientos Min-Heapify, Build-Min-Heap, y Heap-Extract-Min.
20. ¿Es el arreglo ordenado en orden decreciente un max-heap válido? Justifica tu respuesta.
21. **Heap d -ario:** Generaliza el heap binario a un heap d -ario donde cada nodo tiene d hijos. ¿Cuál es la altura de un heap d -ario de n elementos? ¿Cómo afecta d a la complejidad de las operaciones?

Counting Sort y Radix Sort

22. ¿Por qué queremos que el ordenamiento auxiliar usado en Radix Sort sea estable?
23. Suponga que tiene acceso a una subrutina de ordenamiento en tiempo lineal que ordena n elementos en el rango $[1, n^2]$. Describa un algoritmo que ordene n enteros arbitrarios en $O(n)$ usando esta subrutina.
24. **Counting Sort para rangos negativos:** Modifica Counting Sort para manejar enteros en el rango $[-k, k]$.
25. Si tienes n números en el rango $[0, n^3 - 1]$, ¿cuántos dígitos base n tiene cada número? ¿Cuál es la complejidad de Radix Sort en este caso?

2.11.2 Ejercicios de Programación

1. Implementa todos los algoritmos de ordenamiento vistos en este capítulo en tu lenguaje de programación favorito.
2. Escribe un programa que compare experimentalmente el tiempo de ejecución de los diferentes algoritmos para arreglos de diferentes tamaños y distribuciones (aleatorio, ordenado, inversamente ordenado).
3. Implementa una versión iterativa de Merge Sort usando una pila explícita.
4. Implementa Heap Sort usando un min-heap para ordenar en orden ascendente.

2.11.3 Problemas de LeetCode

Los siguientes problemas de LeetCode permiten practicar la aplicación de algoritmos de ordenamiento:

Nivel Fácil

1. **LeetCode 912 - Sort an Array**¹

Dado un arreglo de enteros `nums`, ordénalo en orden ascendente y devuélvelo.

Hint: Implementa Merge Sort o Quick Sort. No uses la función de ordenamiento integrada.

2. **LeetCode 88 - Merge Sorted Array**²

Dados dos arreglos ordenados `nums1` y `nums2`, mezcla `nums2` en `nums1` como un solo arreglo ordenado.

Hint: Usa la técnica de merge de Merge Sort, pero trabajando desde el final hacia el inicio.

3. **LeetCode 217 - Contains Duplicate**³

Dado un arreglo de enteros, determina si contiene duplicados.

Hint: Ordena el arreglo y busca elementos adyacentes iguales.

4. **LeetCode 242 - Valid Anagram**⁴

Dadas dos cadenas `s` y `t`, determina si `t` es un anagrama de `s`.

Hint: Ordena ambas cadenas y compáralas, o usa Counting Sort sobre los caracteres.

Nivel Medio

5. **LeetCode 75 - Sort Colors**⁵

Dado un arreglo con n objetos coloreados de rojo (0), blanco (1) o azul (2), ordénalos in-place.

Hint: Este es el problema de la bandera holandesa. Usa particionamiento similar a Quick Sort con 3 regiones.

6. **LeetCode 148 - Sort List**⁶

Dada la cabeza de una lista enlazada, devuélvela ordenada en orden ascendente.

Hint: Implementa Merge Sort para listas enlazadas. Usa slow / fast pointers para encontrar el punto medio.

7. **LeetCode 179 - Largest Number**⁷

Dado una lista de enteros no negativos, arregla los números para formar el número más grande.

Hint: Define un comparador personalizado: compara $a + b$ vs $b + a$ como strings.

¹<https://leetcode.com/problems/sort-an-array/>

²<https://leetcode.com/problems/merge-sorted-array/>

³<https://leetcode.com/problems/contains-duplicate/>

⁴<https://leetcode.com/problems/valid-anagram/>

⁵<https://leetcode.com/problems/sort-colors/>

⁶<https://leetcode.com/problems/sort-list/>

⁷<https://leetcode.com/problems/largest-number/>

8. **LeetCode 215 - Kth Largest Element in an Array**⁸

Encuentra el k -ésimo elemento más grande en un arreglo desordenado.

Hint: Usa un min-heap de tamaño k , o el algoritmo Quick Select (variante de Quick Sort).

9. **LeetCode 347 - Top K Frequent Elements**⁹

Dado un arreglo de enteros y un entero k , devuelve los k elementos más frecuentes.

Hint: Usa un heap para mantener los k elementos más frecuentes, o Bucket Sort por frecuencia.

10. **LeetCode 56 - Merge Intervals**¹⁰

Dado un arreglo de intervalos, mezcla todos los intervalos superpuestos.

Hint: Ordena los intervalos por tiempo de inicio, luego itera mezclando intervalos superpuestos.

11. **LeetCode 973 - K Closest Points to Origin**¹¹

Dado un arreglo de puntos, encuentra los k puntos más cercanos al origen.

Hint: Usa un max-heap de tamaño k , o Quick Select.

Nivel Difícil

12. **LeetCode 315 - Count of Smaller Numbers After Self**¹²

Dado un arreglo de enteros, devuelve un nuevo arreglo `counts` donde `counts[i]` es el número de elementos más pequeños a la derecha de `nums[i]`.

Hint: Modifica Merge Sort para contar inversiones mientras ordenas.

13. **LeetCode 493 - Reverse Pairs**¹³

Dado un arreglo `nums`, devuelve el número de pares inversos importantes. Un par inverso importante es un par (i, j) donde $i < j$ y $nums[i] > 2 \cdot nums[j]$.

Hint: Similar al conteo de inversiones con Merge Sort, pero con condición modificada.

14. **LeetCode 327 - Count of Range Sum**¹⁴

Dado un arreglo de enteros y un rango $[lower, upper]$, devuelve el número de sumas de rango que caen dentro del rango.

Hint: Usa sumas prefijo y Merge Sort para contar pares que cumplen la condición.

15. **LeetCode 23 - Merge k Sorted Lists**¹⁵

Mezcla k listas enlazadas ordenadas en una sola lista enlazada ordenada.

Hint: Usa un min-heap para siempre obtener el siguiente elemento más pequeño, o aplica la técnica de merge divide-and-conquer.

⁸<https://leetcode.com/problems/kth-largest-element-in-an-array/>

⁹<https://leetcode.com/problems/top-k-frequent-elements/>

¹⁰<https://leetcode.com/problems/merge-intervals/>

¹¹<https://leetcode.com/problems/k-closest-points-to-origin/>

¹²<https://leetcode.com/problems/count-of-smaller-numbers-after-self/>

¹³<https://leetcode.com/problems/reverse-pairs/>

¹⁴<https://leetcode.com/problems/count-of-range-sum/>

¹⁵<https://leetcode.com/problems/merge-k-sorted-lists/>

2.11.4 Problemas Adicionales de Análisis

1. **Límite inferior de ordenamiento:** Demuestra formalmente que cualquier algoritmo de ordenamiento basado en comparaciones requiere al menos $\lceil \lg(n!) \rceil$ comparaciones en el peor caso. Usa la aproximación de Stirling para mostrar que esto es $\Omega(n \lg n)$.
2. **Estabilidad:** Para cada algoritmo de este capítulo, determina si es estable o no. Si no es estable, proporciona un contraejemplo.
3. **Ordenamiento adaptativo:** Un algoritmo es *adaptativo* si su rendimiento mejora cuando la entrada ya está parcialmente ordenada. ¿Cuáles de los algoritmos estudiados son adaptativos?
4. **Trade-offs de espacio:** Compara el uso de memoria adicional de cada algoritmo. ¿Cuál usarías si la memoria es muy limitada?

2.12 Referencias

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4ta ed.). MIT Press. Capítulos 2, 6, 7, 8.
- Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4ta ed.). Addison-Wesley. Capítulo 2.
- Knuth, D. E. (1998). *The Art of Computer Programming, Volume 3: Sorting and Searching* (2da ed.). Addison-Wesley.

2.13 Apéndice: Código de Soporte

Esta sección documenta las funciones de utilidad disponibles en el directorio `src/` del repositorio. Estas funciones están diseñadas para facilitar la experimentación y prueba de los algoritmos de ordenamiento.

2.13.1 Estructura del directorio

```
01-ordenamiento/
+-- src/
    +-- __init__.py
    +-- main.py          # Implementaciones de algoritmos
    +-- test_utils.py    # Utilidades para pruebas
    +-- utils_general.py # Utilidades generales
```

2.13.2 Módulo `test_utils.py`

Este módulo proporciona funciones para generar datos de prueba y medir el rendimiento de los algoritmos.

`generate_random_data(size: int) -> List[int]`

Genera una lista de números enteros aleatorios en el rango $[0, 10000]$.

```
1 from test_utils import generate_random_data
2
3 # Generar 1000 numeros aleatorios
4 datos = generate_random_data(1000)
5 print(f"Primeros 10: {datos[:10]}")
```

- **Parámetro:** `size` – Tamaño de la lista a generar.

- **Retorna:** Lista de enteros aleatorios.
- **Uso típico:** Probar algoritmos con entrada aleatoria (caso promedio).

generate_nearly_sorted(size: int, swaps: int = None) ->List[int]

Genera una lista casi ordenada realizando algunos intercambios aleatorios sobre una lista ordenada.

```
1 from test_utils import generate_nearly_sorted
2
3 # Lista casi ordenada con pocos intercambios
4 datos = generate_nearly_sorted(1000, swaps=10)
```

- **Parámetros:**
 - size – Tamaño de la lista.
 - swaps – Número de intercambios (default: size // 10).
- **Retorna:** Lista casi ordenada.
- **Uso típico:** Evaluar algoritmos adaptativos como Insertion Sort.

generate_reversed(size: int) ->List[int]

Genera una lista ordenada en orden inverso (descendente).

```
1 from test_utils import generate_reversed
2
3 # Lista [1000, 999, 998, ..., 2, 1]
4 datos = generate_reversed(1000)
```

- **Parámetro:** size – Tamaño de la lista.
- **Retorna:** Lista ordenada inversamente.
- **Uso típico:** Probar el peor caso de algoritmos como Quick Sort (sin aleatorización).

measure_time(func: Callable, arr: List[int]) ->float

Mide el tiempo de ejecución de un algoritmo de ordenamiento.

```
1 from test_utils import measure_time, generate_random_data
2 from main import merge_sort, quick_sort
3
4 datos = generate_random_data(10000)
5
6 # Medir tiempo de Merge Sort
7 tiempo_merge = measure_time(merge_sort, datos)
8 print(f"Merge Sort: {tiempo_merge:.4f} segundos")
9
10 # Medir tiempo de Quick Sort
11 tiempo_quick = measure_time(quick_sort, datos)
12 print(f"Quick Sort: {tiempo_quick:.4f} segundos")
```

■ **Parámetros:**

- `func` – Función de ordenamiento a medir.
- `arr` – Lista a ordenar.

■ **Retorna:** Tiempo de ejecución en segundos.

■ **Nota:** La función crea una copia del arreglo para no modificar el original.

2.13.3 Módulo `main.py`

Contiene los esqueletos de los algoritmos de ordenamiento para que los implementes.

Función	Descripción
<code>bubble_sort(arr)</code>	Bubble Sort básico
<code>merge_sort(arr)</code>	Merge Sort recursivo
<code>quick_sort(arr)</code>	Quick Sort con pivote
<code>heapify(arr, n, i)</code>	Auxiliar para Heap Sort
<code>heap_sort(arr)</code>	Heap Sort usando max-heap
<code>counting_sort(arr)</code>	Counting Sort para enteros
<code>counting_sort_for_radix(arr, exp)</code>	Counting Sort modificado
<code>radix_sort(arr)</code>	Radix Sort LSD

Cuadro 2.2: Funciones disponibles en `main.py`

2.13.4 Ejemplo: Comparación experimental

El siguiente script compara todos los algoritmos:

```

1 from test_utils import (generate_random_data, generate_nearly_sorted,
2                           generate_reversed, measure_time)
3 from main import (bubble_sort, merge_sort, quick_sort,
4                   heap_sort, counting_sort, radix_sort)
5
6 # Configuración
7 SIZES = [100, 500, 1000, 5000]
8 algorithms = {
9     'Bubble Sort': bubble_sort,
10    'Merge Sort': merge_sort,
11    'Quick Sort': quick_sort,
12    'Heap Sort': heap_sort,
13 }
14
15 # Experimento
16 for size in SIZES:
17     print(f"\n=== Tamaño: {size} ===")
18     datos = generate_random_data(size)
19
20     for name, func in algorithms.items():
21         tiempo = measure_time(func, datos)
22         print(f"{name}: {tiempo:.4f}s")

```

2.13.5 Verificación de correctitud

Para verificar que tu implementación es correcta:

```
1 from test_utils import generate_random_data
2 from main import merge_sort # o cualquier algoritmo
3
4 def verificar_ordenamiento(func, n=100, pruebas=10):
5     """Verifica que un algoritmo ordena correctamente."""
6     for _ in range(pruebas):
7         datos = generate_random_data(n)
8         esperado = sorted(datos)
9         resultado = func(datos.copy())
10        assert resultado == esperado, "Error en ordenamiento!"
11        print(f"{func.__name__} paso todas las pruebas")
12
13 verificar_ordenamiento(merge_sort)
```

CAPÍTULO 3

Algoritmos de Búsqueda

La búsqueda es una de las operaciones más fundamentales en ciencias de la computación. Desde buscar un nombre en una lista de contactos hasta encontrar la ruta más corta en un mapa, los algoritmos de búsqueda son omnipresentes. En este capítulo estudiaremos técnicas para buscar elementos en colecciones de datos, comenzando con métodos simples y progresando hacia estructuras más sofisticadas como las tablas hash.

3.1 El problema de la búsqueda

Problema de la Búsqueda

Entrada: Una secuencia de n números $A = \langle a_1, a_2, \dots, a_n \rangle$ y un valor v .

Salida: Un índice i tal que $v = A[i]$, o el valor especial NIL si v no aparece en A .

La eficiencia de un algoritmo de búsqueda depende crucialmente de:

- Si los datos están **ordenados** o no
- La **estructura de datos** utilizada para almacenarlos
- Si necesitamos búsquedas **repetidas** sobre los mismos datos

3.2 Búsqueda Lineal (Secuencial)

La **búsqueda lineal** es el método más simple: examinar cada elemento de la secuencia hasta encontrar el valor buscado o agotar todos los elementos.

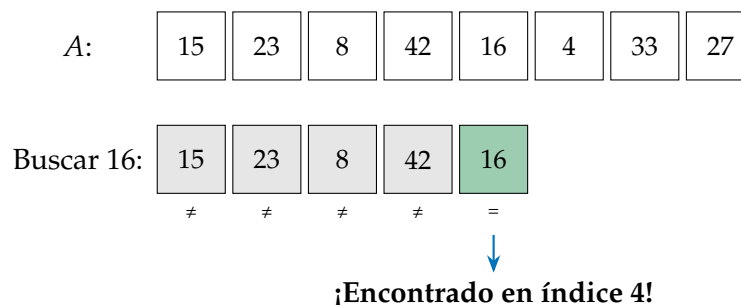


Figura 3.1: Búsqueda lineal: se examina cada elemento secuencialmente hasta encontrar el valor 16.

3.2.1 Pseudocódigo

Algorithm 13 Linear-Search(A, n, v)

```

1: for  $i = 1$  to  $n$  do
2:   if  $A[i] = v$  then
3:     return  $i$ 
4:   end if
5: end for
6: return NIL

```

3.2.2 Análisis de complejidad

- **Mejor caso:** $O(1)$ — el elemento está en la primera posición.
- **Peor caso:** $O(n)$ — el elemento está en la última posición o no existe.
- **Caso promedio:** $O(n)$ — en promedio, examina $n/2$ elementos.

3.2.3 Invariante de ciclo

Invariante: Al inicio de cada iteración del ciclo **for**, el subarreglo $A[1..i - 1]$ no contiene el valor v .

3.3 Búsqueda Binaria

Cuando los datos están **ordenados**, podemos hacer mucho mejor que la búsqueda lineal. La **búsqueda binaria** aprovecha el orden para descartar la mitad de los elementos restantes en cada paso.



Figura 3.2: Búsqueda binaria: en cada iteración se descarta la mitad del espacio de búsqueda.

3.3.1 Pseudocódigo iterativo

Algorithm 14 Binary-Search($A, v, low, high$)

```

1: while  $low \leq high$  do
2:    $mid = \lfloor (low + high)/2 \rfloor$ 
3:   if  $v = A[mid]$  then
4:     return  $mid$ 
5:   else if  $v > A[mid]$  then
6:      $low = mid + 1$ 
7:   else
8:      $high = mid - 1$ 
9:   end if
10: end while
11: return NIL

```

3.3.2 Pseudocódigo recursivo

Algorithm 15 Binary-Search-Recursive($A, v, low, high$)

```

1: if  $low > high$  then
2:   return NIL
3: end if
4:  $mid = \lfloor (low + high)/2 \rfloor$ 
5: if  $v = A[mid]$  then
6:   return  $mid$ 
7: else if  $v > A[mid]$  then
8:   return Binary-Search-Recursive( $A, v, mid + 1, high$ )
9: else
10:  return Binary-Search-Recursive( $A, v, low, mid - 1$ )
11: end if

```

3.3.3 Análisis de complejidad

Teorema 3.1. La búsqueda binaria tiene complejidad $O(\lg n)$ en el peor caso.

Demostración. En cada iteración, el tamaño del espacio de búsqueda se reduce a la mitad. Si comenzamos con n elementos:

- Después de 1 iteración: $n/2$ elementos
- Después de 2 iteraciones: $n/4$ elementos
- Después de k iteraciones: $n/2^k$ elementos

El algoritmo termina cuando $n/2^k \leq 1$, es decir, cuando $k \geq \lg n$.

Por lo tanto, el número máximo de iteraciones es $\lceil \lg n \rceil + 1 = O(\lg n)$. □

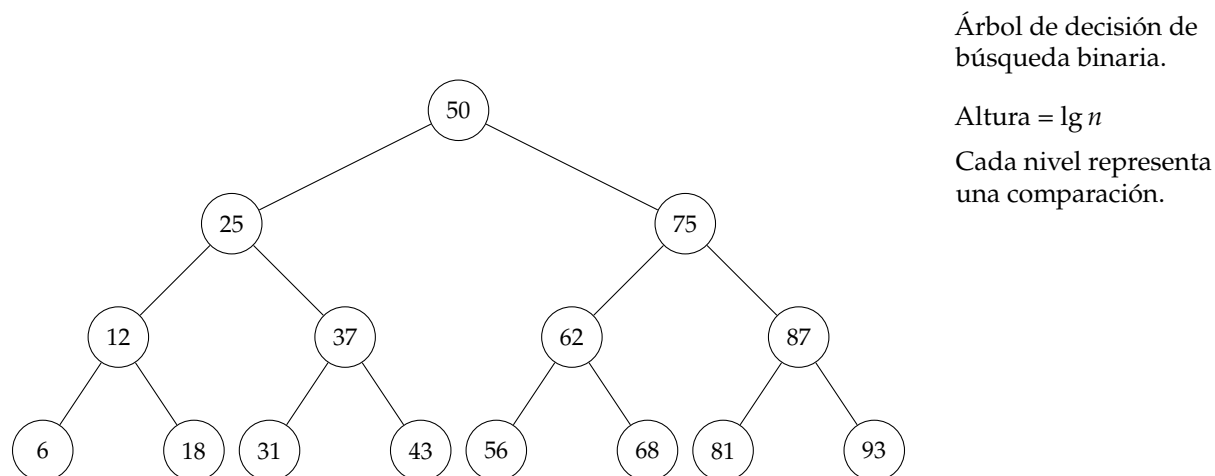


Figura 3.3: La búsqueda binaria puede visualizarse como un árbol binario de búsqueda. La altura del árbol es $\lg n$.

Tamaño n	Búsqueda Lineal	Búsqueda Binaria
10	10	4
100	100	7
1,000	1,000	10
1,000,000	1,000,000	20
1,000,000,000	1,000,000,000	30

Cuadro 3.1: Comparaciones en el peor caso para búsqueda lineal vs. binaria.

3.3.4 Comparación: Lineal vs. Binaria

3.4 Tablas Hash

Las tablas hash permiten búsquedas en tiempo $O(1)$ *en promedio*, lo cual es incluso mejor que la búsqueda binaria. La idea es usar una **función hash** para calcular directamente la posición donde debería estar un elemento.

Definición 3.1 (Tabla Hash). Una **tabla hash** es una estructura de datos que implementa un **diccionario**: una colección de pares (clave, valor) con operaciones de inserción, eliminación y búsqueda.

3.4.1 Direccionamiento directo

Si las claves son enteros en un rango pequeño $[0, m - 1]$, podemos usar un arreglo $T[0..m - 1]$ donde $T[k]$ almacena el elemento con clave k .

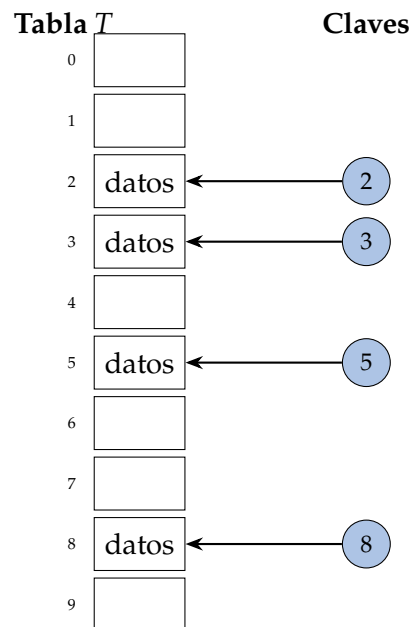


Figura 3.4: Direccionamiento directo: la clave k se almacena directamente en $T[k]$.

Problema: Si el universo de claves es grande (ej. números de 32 bits), necesitaríamos un arreglo de 2^{32} posiciones — ¡imposible!

3.4.2 Funciones hash

Una **función hash** $h : U \rightarrow \{0, 1, \dots, m - 1\}$ mapea el universo de claves U a las posiciones de una tabla de tamaño m .

Método de la división

$$h(k) = k \mod m$$

Donde m es el tamaño de la tabla. Funciona mejor si m es un número primo no cercano a una potencia de 2.

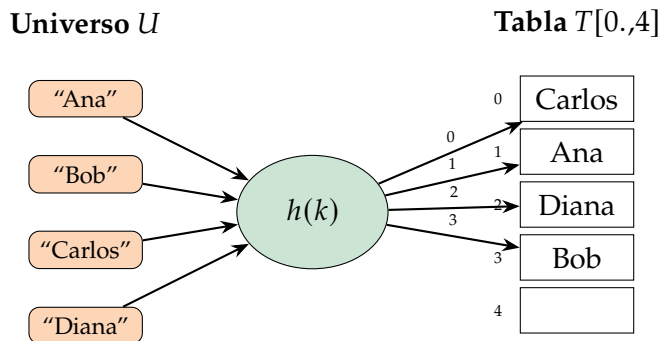


Figura 3.5: Una función hash mapea claves del universo U a posiciones en la tabla T .

Método de la multiplicación

$$h(k) = \lfloor m(kA \bmod 1) \rfloor$$

Donde A es una constante, $0 < A < 1$. Knuth sugiere $A \approx (\sqrt{5} - 1)/2 = 0,6180339887...$ (el inverso de la razón áurea).

3.4.3 Colisiones

Una **colisión** ocurre cuando dos claves diferentes $k_1 \neq k_2$ tienen el mismo valor hash: $h(k_1) = h(k_2)$.

Por el *principio del palomar*, si tenemos más de m claves, habrá colisiones. Incluso con menos claves, las colisiones son probables (ver la *paradoja del cumpleaños*).

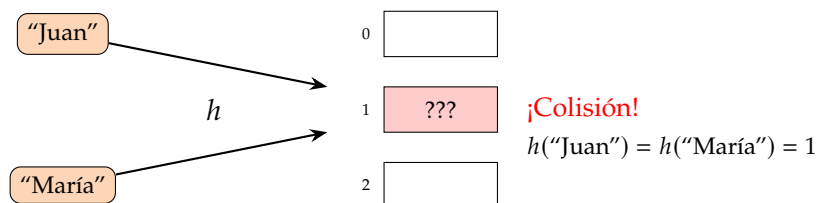


Figura 3.6: Una colisión ocurre cuando dos claves diferentes producen el mismo índice.

3.5 Resolución de colisiones

3.5.1 Encadenamiento (Chaining)

En el **encadenamiento**, cada posición de la tabla contiene una lista enlazada de todos los elementos que colisionan en esa posición.

Operaciones con encadenamiento

Algorithm 16 Chained-Hash-Insert(T, x)

1: insertar x al inicio de la lista $T[h(x.key)]$

Algorithm 17 Chained-Hash-Search(T, k)

1: buscar un elemento con clave k en la lista $T[h(k)]$

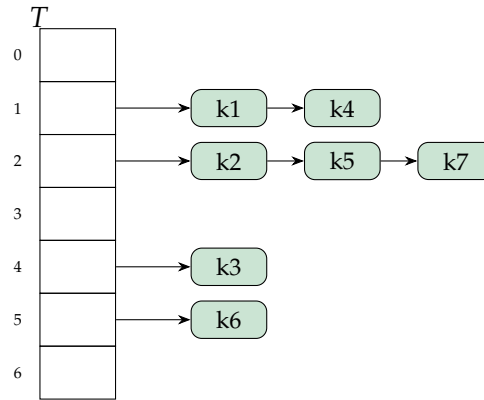


Figura 3.7: Resolución de colisiones por encadenamiento: cada slot contiene una lista enlazada.

Algorithm 18 Chained-Hash-Delete(T, x)

1: eliminar x de la lista $T[h(x.key)]$

Análisis del encadenamiento

Definición 3.2 (Factor de carga). *El factor de carga de una tabla hash con n elementos y m slots es:*

$$\alpha = \frac{n}{m}$$

Teorema 3.2. *En una tabla hash con encadenamiento, bajo la suposición de hashing uniforme simple, una búsqueda sin éxito toma tiempo $\Theta(1 + \alpha)$ en promedio.*

Teorema 3.3. *En una tabla hash con encadenamiento, bajo la suposición de hashing uniforme simple, una búsqueda con éxito toma tiempo $\Theta(1 + \alpha)$ en promedio.*

Si $n = O(m)$, entonces $\alpha = O(1)$, y las operaciones toman tiempo $O(1)$ en promedio.

3.5.2 Direccionamiento abierto (Open Addressing)

En el **direccionamiento abierto**, todos los elementos se almacenan en la tabla misma. Cuando hay una colisión, se busca otra posición disponible mediante una secuencia de **sondeo** (probing).

Sondeo lineal

La secuencia de sondeo es:

$$h(k, i) = (h'(k) + i) \mod m \quad \text{para } i = 0, 1, 2, \dots, m - 1$$

Problema: *Agrupamiento primario* — las celdas ocupadas tienden a formar bloques largos.

Sondeo cuadrático

$$h(k, i) = (h'(k) + c_1 i + c_2 i^2) \mod m$$

Reduce el agrupamiento primario pero puede causar *agrupamiento secundario*.

Doble hashing

$$h(k, i) = (h_1(k) + i \cdot h_2(k)) \mod m$$

Usa dos funciones hash. Produce una mejor distribución.

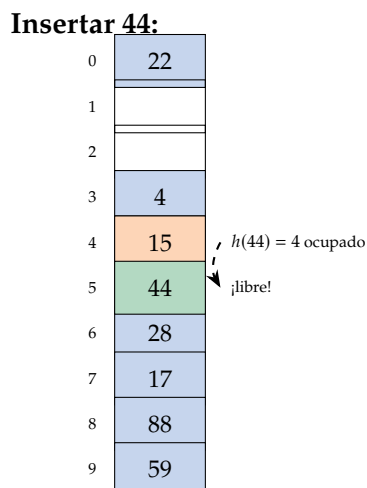


Figura 3.8: Direccionamiento abierto con sondeo lineal: si $h(44) = 4$ está ocupado, se prueba 5.

Pseudocódigo

Algorithm 19 Hash-Insert(T, k)

```

1:  $i = 0$ 
2: repeat
3:    $j = h(k, i)$ 
4:   if  $T[j] = \text{NIL}$  then
5:      $T[j] = k$ 
6:     return  $j$ 
7:   else
8:      $i = i + 1$ 
9:   end if
10: until  $i = m$ 
11: error "tabla llena"

```

Algorithm 20 Hash-Search(T, k)

```

1:  $i = 0$ 
2: repeat
3:    $j = h(k, i)$ 
4:   if  $T[j] = k$  then
5:     return  $j$ 
6:   end if
7:    $i = i + 1$ 
8: until  $T[j] = \text{NIL}$  or  $i = m$ 
9: return NIL

```

3.6 Hashing perfecto

Cuando el conjunto de claves es **estático** (no cambia), podemos construir una tabla hash con búsqueda $O(1)$ en el *peor caso* usando **hashing perfecto**.

La idea es usar dos niveles de hashing:

1. El primer nivel usa una función hash para distribuir las claves en m slots.
2. Cada slot que tiene colisiones usa una *segunda* tabla hash con una función elegida para evitar colisiones.

Teorema 3.4. Si usamos tablas secundarias de tamaño n_j^2 (donde n_j es el número de claves que colisionan en el slot j), el espacio total esperado es $O(n)$.

3.7 Aplicaciones de tablas hash

Las tablas hash son ubicuas en la práctica:

- **Diccionarios y conjuntos:** Implementación de `dict` en Python, `HashMap` en Java.
- **Cachés:** Almacenar resultados de cálculos costosos.
- **Bases de datos:** Índices hash para búsqueda rápida.
- **Compiladores:** Tablas de símbolos para variables y funciones.
- **Detección de duplicados:** Verificar si un elemento ya fue visto.
- **Conteo de frecuencias:** Contar ocurrencias de palabras, etc.

3.8 Comparación de métodos de búsqueda

Método	Búsqueda	Inserción	Requisitos
Búsqueda lineal	$O(n)$	$O(1)$	Ninguno
Búsqueda binaria	$O(\lg n)$	$O(n)$	Datos ordenados
Tabla hash (prom.)	$O(1)$	$O(1)$	Buena función hash
Tabla hash (peor)	$O(n)$	$O(n)$	—

Cuadro 3.2: Comparación de complejidades de métodos de búsqueda.

3.9 Ejercicios

1. Escriba el pseudocódigo de búsqueda lineal con centinela y analice su complejidad.
2. Modifique la búsqueda binaria para que devuelva el índice donde *debería* insertarse el elemento si no existe.
3. Dada una tabla hash de tamaño $m = 11$ y la función hash $h(k) = k \bmod 11$, muestre el resultado de insertar las claves 10, 22, 31, 4, 15, 28, 17, 88, 59 usando:
 - a) Encadenamiento
 - b) Sondeo lineal
 - c) Sondeo cuadrático con $c_1 = 1, c_2 = 3$
4. Demuestre que si $|U| > nm$, existe un subconjunto de n claves que todas hacen hash al mismo slot.

5. ¿Cuál es la probabilidad de que al insertar k elementos en una tabla hash de tamaño n no haya colisiones? (Paradoja del cumpleaños)
6. Implemente una función hash para strings usando el método de Horner.
7. Diseñe una estructura de datos que soporte las siguientes operaciones en $O(1)$ promedio:
 - `Insert(x)`: Insertar elemento x
 - `Delete(x)`: Eliminar elemento x
 - `GetRandom()`: Devolver un elemento aleatorio uniforme
8. El profesor sugiere usar $h(k) = k \bmod m$ donde $m = 2^p - 1$ y k es un string interpretado como número en base 2^p . ¿Por qué es esta una mala idea?
9. Considere una tabla hash con encadenamiento donde las listas están ordenadas. ¿Cómo afecta esto los tiempos de búsqueda con éxito y sin éxito?
10. Diseñe un algoritmo que, dado un arreglo de n enteros, determine si existen dos elementos cuya suma sea un valor dado S . Use una tabla hash para lograr $O(n)$ en promedio.

3.10 Referencias

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4ta ed.). MIT Press. Capítulos 11, 12.
- Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4ta ed.). Addison-Wesley. Capítulo 3.
- Knuth, D. E. (1998). *The Art of Computer Programming, Volume 3: Sorting and Searching* (2da ed.). Addison-Wesley. Capítulo 6.

3.11 Apendice: Código de Soporte

Esta sección documenta las funciones de utilidad disponibles en el directorio `src/` del repositorio para algoritmos de búsqueda.

3.11.1 Estructura del directorio

`02-busqueda/`

`+-+ src/`

`+-+ main.py # Implementaciones de algoritmos de busqueda`

3.11.2 Módulo `main.py`

Este módulo contiene las implementaciones de los algoritmos de búsqueda estudiados en este capítulo.

`busqueda_lineal(lista, objetivo) ->int`

Implementación iterativa de la búsqueda lineal.

```

1 from main import busqueda_lineal
2
3 datos = [4, 2, 7, 1, 9, 3]
4 indice = busqueda_lineal(datos, 7)
5 print(f"Encontrado en indice: {indice}") # Output: 2

```

- **Parametros:**

- lista – Lista de elementos a buscar.
- objetivo – Elemento a encontrar.

- **Retorna:** Indice del elemento si se encuentra, -1 si no existe.

- **Complejidad:** $O(n)$ en el peor caso.

busqueda_lineal_rekursiva(lista, objetivo, indice=0) ->int

Version recursiva de la busqueda lineal.

```
1 from main import busqueda_lineal_rekursiva
2
3 datos = [4, 2, 7, 1, 9, 3]
4 indice = busqueda_lineal_rekursiva(datos, 9)
5 print(f"Encontrado en indice: {indice}") # Output: 4
```

- **Parametros:**

- lista – Lista de elementos.
- objetivo – Elemento a encontrar.
- indice – Indice inicial (default: 0).

- **Retorna:** Indice del elemento o -1 .

- **Nota:** Usa $O(n)$ espacio en la pila de llamadas.

busqueda_binaria(lista, objetivo) ->int

Implementacion iterativa de la busqueda binaria. **Requiere lista ordenada.**

```
1 from main import busqueda_binaria
2
3 datos = [1, 2, 3, 4, 7, 9] # DEBE estar ordenada
4 indice = busqueda_binaria(datos, 7)
5 print(f"Encontrado en indice: {indice}") # Output: 4
```

- **Parametros:**

- lista – Lista **ordenada** de elementos.
- objetivo – Elemento a encontrar.

- **Retorna:** Indice del elemento o -1 .

- **Complejidad:** $O(\lg n)$ tiempo, $O(1)$ espacio.

- **Precondicion:** La lista debe estar ordenada.

busqueda_binaria_recursiva(lista, objetivo, inicio=0, fin=None) ->int

Version recursiva de la busqueda binaria.

```

1 from main import busqueda_binaria_recursiva
2
3 datos = [1, 2, 3, 4, 7, 9]
4 indice = busqueda_binaria_recursiva(datos, 4)
5 print(f"Encontrado en indice: {indice}") # Output: 3
6
7 # Buscar en un subrango
8 indice = busqueda_binaria_recursiva(datos, 7, inicio=2, fin=5)

```

■ **Parametros:**

- lista – Lista ordenada.
- objetivo – Elemento a encontrar.
- inicio – Indice inicial del rango (default: 0).
- fin – Indice final del rango (default: len(lista) - 1).

■ **Retorna:** Indice del elemento o -1.

■ **Complejidad:** $O(\lg n)$ tiempo, $O(\lg n)$ espacio en pila.

3.11.3 Comparacion de algoritmos

Funcion	Tiempo	Espacio	Requisito
busqueda_lineal	$O(n)$	$O(1)$	Ninguno
busqueda_lineal_recursiva	$O(n)$	$O(n)$	Ninguno
busqueda_binaria	$O(\lg n)$	$O(1)$	Lista ordenada
busqueda_binaria_recursiva	$O(\lg n)$	$O(\lg n)$	Lista ordenada

Cuadro 3.3: Comparacion de funciones de busqueda

3.11.4 Ejemplo: Comparacion de rendimiento

```

1 import time
2 from main import busqueda_lineal, busqueda_binaria
3
4 def medir_busqueda(func, lista, objetivo, repeticiones=1000):
5     inicio = time.time()
6     for _ in range(repeticiones):
7         func(lista, objetivo)
8     return (time.time() - inicio) / repeticiones
9
10 # Crear lista grande ordenada
11 n = 1000000
12 datos = list(range(n))
13 objetivo = n - 1 # Peor caso para lineal
14
15 t_lineal = medir_busqueda(busqueda_lineal, datos, objetivo)

```

```
16 t_binaria = medir_busqueda(busqueda_binaria, datos, objetivo)
17
18 print(f"Busqueda lineal: {t_lineal*1000:.4f} ms")
19 print(f"Busqueda binaria: {t_binaria*1000:.4f} ms")
20 print(f"Speedup: {t_lineal/t_binaria:.1f}x")
```

3.11.5 Errores comunes

1. Usar búsqueda binaria en lista no ordenada:

```
1 # INCORRECTO - lista no ordenada
2 datos = [5, 2, 8, 1, 9]
3 busqueda_binaria(datos, 8) # Resultado impredecible!
4
5 # CORRECTO
6 datos_ordenados = sorted(datos)
7 busqueda_binaria(datos_ordenados, 8)
```

2. Overflow en cálculo de punto medio:

```
1 # En lenguajes con enteros de tamaño fijo:
2 # INCORRECTO (puede overflow)
3 medio = (inicio + fin) / 2
4
5 # CORRECTO
6 medio = inicio + (fin - inicio) // 2
```

Bibliografía
